

File Systems

Table of Contents

1. [What Is a File System?](#)
2. [Files](#)
3. [File Operations](#)
4. [File Attributes and Metadata](#)
5. [File Access Methods](#)
6. [Directory Structures](#)
7. [File System Mounting](#)
8. [Disk Blocks and File Allocation](#)
9. [Contiguous Allocation](#)
10. [Linked Allocation](#)
11. [FAT: File Allocation Table](#)
12. [Indexed Allocation](#)
13. [Inodes](#)
14. [Free-Space Management](#)
15. [Journaling](#)
16. [Permissions and Protection](#)
17. [File System Implementation Layers](#)
18. [Reliability and Recovery](#)
19. [Performance Considerations](#)
20. [Common File Systems](#)
21. [Important Interview Comparisons](#)
22. [Frequently Asked Placement Questions](#)
23. [Quick Revision Sheet](#)

What Is a File System?

A file system is the part of the operating system responsible for storing, organizing, naming, retrieving, protecting, and managing data on storage devices.

Storage devices such as HDDs, SSDs, pen drives, memory cards, and optical disks store raw blocks of data. A file system gives structure to those blocks so users and programs can work with meaningful objects such as files and directories.

Without a file system, storage would look like one large sequence of blocks. The operating system would not know:

- Which blocks belong to which file.
- Where a file starts and ends.
- Which files are inside which directory.
- Which user owns a file.
- Which users can read, write, or execute a file.
- Which blocks are free and which are already used.

- How to recover data after a crash.

In simple words:

A file system is an abstraction over physical storage that allows data to be stored and accessed using files and directories.

Main Responsibilities

A file system handles:

- File creation, deletion, reading, writing, renaming, and truncation.
- Directory creation, deletion, traversal, and lookup.
- Mapping logical file offsets to physical disk blocks.
- Managing free and allocated disk space.
- Maintaining metadata such as file size, owner, permissions, timestamps, and block locations.
- Enforcing access control and protection.
- Supporting crash recovery using techniques such as journaling.
- Improving performance using caching, buffering, prefetching, and block allocation strategies.

Why File Systems Are Needed

Programs usually do not want to think in terms of sectors, tracks, cylinders, flash pages, or physical addresses. They want simple operations such as:

```
open("notes.txt")
read(file_descriptor)
write(file_descriptor)
close(file_descriptor)
```

The file system hides hardware complexity and provides a uniform interface.

File System vs Storage Device

Concept	Meaning
Storage device	Physical hardware that stores bits
Disk block / sector	Fixed-size storage unit on the device
File system	OS structure that organizes storage into files and directories
File	Named collection of related data
Directory	Special file that maps names to files or other directories

Example:

An SSD stores raw pages and blocks. NTFS, ext4, FAT32, or APFS organizes those raw areas into files such as `resume.pdf`, `main.c`, and `photo.jpg`.

Files

A file is a named collection of related information stored on secondary storage.

From a user's point of view, a file may be:

- A text document.
- A source code file.
- An executable program.
- An image.
- A video.
- A database.
- A compressed archive.

From an operating system's point of view, a file is usually a sequence of bytes or records, along with metadata.

File as an Abstraction

The file abstraction provides:

- A name.
- A stable storage location.
- A logical sequence of bytes.
- Operations to access and modify data.
- Metadata and protection information.

The OS hides where the file is physically stored.

For example, a file may be split into many non-contiguous disk blocks, but the user sees it as one continuous object.

File Types

Common file types include:

File Type	Example	Meaning
Regular file	<code>.txt</code> , <code>.c</code> , <code>.jpg</code>	Stores user data
Directory file	Folder	Stores mappings of filenames to file metadata
Executable file	<code>.exe</code> , ELF binary	Contains machine instructions
Device file	<code>/dev/sda</code> , <code>/dev/null</code>	Represents hardware or virtual devices
Symbolic link	Shortcut-like file	Points to another file path
Pipe/FIFO	Named pipe	Used for inter-process communication
Socket file	Unix socket	Used for local communication between processes

File Naming

Different operating systems have different naming rules.

Examples:

- Windows uses paths such as `C:\Users\Student\file.txt`.
- Unix/Linux uses paths such as `/home/student/file.txt`.

Common file naming considerations:

- Maximum filename length.
- Allowed characters.
- Case sensitivity.
- Extension conventions.
- Reserved names.

Linux file systems are generally case-sensitive:

```
notes.txt
Notes.txt
NOTES.txt
```

These can be three different files.

Windows file systems are usually case-insensitive by default, so `notes.txt` and `Notes.txt` usually refer to the same file.

File Extensions

A file extension is the suffix after the last dot in a filename.

Examples:

- `main.c`
- `index.html`
- `resume.pdf`
- `archive.zip`

Extensions help users and applications identify the likely file type, but the OS may not always depend on them.

Unix-like systems often identify file type using metadata, magic numbers, or content inspection rather than just extensions.

File Operations

File operations are system calls or API functions used by programs to manipulate files.

The common operations are:

- Create
- Open
- Read

- Write
- Seek
- Close
- Delete
- Truncate
- Rename
- Append
- Get attributes
- Set attributes

Create

Creates a new file and adds an entry for it in the directory.

Steps usually involved:

1. Check whether the file already exists.
2. Check whether the user has permission to create a file in the directory.
3. Allocate metadata structure such as inode or file control block.
4. Add a directory entry mapping filename to metadata.
5. Initialize file size to zero.

Example:

```
int fd = creat("data.txt", 0644);
```

Open

Before reading or writing a file, a process usually opens it.

Opening a file:

- Checks permissions.
- Locates the file's metadata.
- Creates an entry in the system-wide open-file table.
- Creates an entry in the process's file descriptor table.
- Returns a file descriptor or handle.

Example:

```
int fd = open("data.txt", O_RDONLY);
```

Why opening is needed:

- Searching directory structures repeatedly would be slow.
- The OS maintains current file position.
- The OS tracks access mode.

- The OS can manage sharing and locking.

File Descriptor

A file descriptor is a small integer used by a process to refer to an open file.

In Unix-like systems:

File Descriptor	Meaning
0	Standard input
1	Standard output
2	Standard error

Example:

```
read(fd, buffer, size);
```

Here, `fd` tells the OS which open file should be read.

Read

Reads data from an open file into memory.

Steps:

1. Check that the file was opened for reading.
2. Use the current file offset.
3. Convert logical file offset to physical block address.
4. Load block from disk or cache.
5. Copy data into user buffer.
6. Update current file offset.

Example:

```
ssize_t n = read(fd, buffer, 100);
```

Write

Writes data from memory to an open file.

Steps:

1. Check that the file was opened for writing.
2. Use the current file offset.
3. Allocate new blocks if needed.
4. Copy data from user buffer to kernel buffer/page cache.

5. Eventually write modified blocks to disk.
6. Update file size and timestamps if required.

Example:

```
write(fd, "hello", 5);
```

Writes may be buffered. This means `write()` can return before data is physically stored on disk.

Seek

Changes the current file position.

Example:

```
lseek(fd, 0, SEEK_SET);
```

Common seek modes:

Mode	Meaning
<code>SEEK_SET</code>	Offset from beginning
<code>SEEK_CUR</code>	Offset from current position
<code>SEEK_END</code>	Offset from end

Seek is useful for random access.

Example:

To read the 1000th byte, the process can seek directly to byte 999 instead of reading all previous bytes.

Close

Closes an open file.

Steps:

1. Remove file descriptor entry from the process table.
2. Decrement open count in system-wide open-file table.
3. Flush buffers if needed.
4. Release locks if any.
5. Free kernel structures if no process is using the file.

Example:

```
close(fd);
```

Delete

Deletes a file from a directory.

In Unix-like systems, deleting a file usually means unlinking its directory entry.

Important point:

If a process has the file open, the file's data may remain on disk until the last open reference is closed.

Example:

```
unlink("data.txt");
```

Truncate

Truncation changes the file size, often to zero.

Example:

```
truncate("log.txt", 0);
```

If a file is truncated:

- Its old data blocks may be freed.
- Its size metadata is updated.
- The filename and permissions can remain unchanged.

Rename

Changes the name or path of a file.

Example:

```
rename("old.txt", "new.txt");
```

Within the same file system, rename is often atomic. This makes it useful for safe file updates.

Example safe update pattern:

1. Write new contents to `file.tmp`.
2. Flush `file.tmp`.
3. Rename `file.tmp` to `file`.

This avoids leaving a half-written final file.

Append

Append writes data at the end of the file.

With append mode, the OS ensures each write goes to the current end of file.

Example:

```
open("log.txt", O_WRONLY | O_APPEND);
```

Append mode is useful for log files.

Get and Set Attributes

The OS allows programs to inspect or modify metadata.

Examples of file attributes:

- Size
- Owner
- Permissions
- Creation time
- Modification time
- Access time
- File type
- Link count

Example command:

```
stat file.txt
```

File Attributes and Metadata

Metadata means "data about data."

For a file, metadata describes the file but is not the actual file content.

Common file metadata:

Attribute	Meaning
Name	Human-readable filename
Identifier	Internal unique file identifier such as inode number
Type	Regular file, directory, device file, symbolic link, etc.

Attribute	Meaning
Location	Pointers to disk blocks
Size	Current file size
Protection	Read/write/execute permissions
Owner	User who owns the file
Group	Group associated with the file
Timestamps	Creation, access, modification, metadata-change times
Link count	Number of directory entries pointing to the file

File Control Block

A File Control Block, or FCB, is an OS data structure that stores file metadata.

In Unix-like systems, the inode is the main file control block.

In other systems, similar structures may be called:

- FCB
- MFT record
- Catalog record
- File metadata record

The exact name differs, but the idea is the same: the file system needs a metadata structure to describe each file.

File Access Methods

File access method defines how data inside a file is read or written.

The three common access methods are:

- Sequential access
- Direct/random access
- Indexed access

Sequential Access

In sequential access, data is processed in order from beginning to end.

Example:

```
Read record 1
Read record 2
Read record 3
...
```

Used in:

- Text files
- Logs
- Media streaming
- Tape drives
- Simple file processing

Advantages:

- Simple.
- Efficient for full-file reading.
- Good for streaming workloads.

Disadvantages:

- Slow if you need data near the end.
- Not suitable for frequent random lookups.

Direct or Random Access

In direct access, a process can jump to any position in the file.

Example:

```
Go directly to byte offset 50000
Read 100 bytes
```

Used in:

- Databases
- Large binary files
- Virtual memory swap files
- Index-based data stores

Advantages:

- Fast access to arbitrary locations.
- Useful for structured records.

Disadvantages:

- More complex.

- May cause many disk seeks on HDDs if access is scattered.

Indexed Access

Indexed access uses an index to locate records quickly.

Example:

An employee file may have an index:

```
Employee ID 101 -> Block 5  
Employee ID 205 -> Block 17  
Employee ID 309 -> Block 26
```

Used in:

- Database systems
- File systems with indexed allocation
- Search systems

Advantages:

- Fast lookup.
- Good for large files with structured records.

Disadvantages:

- Extra storage for index.
- Index must be maintained during insertions, deletions, and updates.

Directory Structures

A directory is a special file used to organize files.

It maps filenames to file metadata references.

In Unix-like systems, a directory maps:

```
filename -> inode number
```

In simple terms, a directory is like a table of contents for files.

Directory Operations

Common directory operations:

- Create a directory.
- Delete a directory.

- Open a directory.
- Read directory entries.
- Rename files or directories.
- Search for a file.
- List directory contents.
- Traverse a path.

Example commands:

```
mkdir notes  
ls  
cd notes  
rmdir notes
```

Pathnames

A pathname identifies a file's location in the directory hierarchy.

Types:

- Absolute path
- Relative path

Absolute path starts from the root directory.

Example:

```
/home/student/os/file-systems.md
```

Relative path starts from the current working directory.

Example:

```
../os/file-systems.md
```

Single-Level Directory

In a single-level directory, all files are kept in one directory.

Structure:

```
root
|-- file1.txt
|-- file2.txt
|-- program.c
```

Advantages:

- Very simple.
- Easy to implement.

Disadvantages:

- Filename conflicts are common.
- No grouping of related files.
- Poor for multiple users.
- Searching becomes slow as files increase.

Placement point:

Single-level directory is suitable only for very small systems.

Two-Level Directory

In a two-level directory, each user gets a separate directory.

Structure:

```
Master File Directory
|-- user1
|   |-- a.txt
|   |-- b.txt
|-- user2
|   |-- a.txt
|   |-- c.txt
```

Advantages:

- Different users can have files with the same name.
- Better isolation than single-level directory.
- Easier user-based organization.

Disadvantages:

- Still limited grouping inside each user directory.
- Sharing between users can be inconvenient.
- Not flexible enough for complex systems.

Tree-Structured Directory

In a tree-structured directory, directories can contain files and subdirectories.

Structure:

```
/
|-- home
|   |-- student
|       |-- os
|       |-- dbms
|-- bin
|-- etc
```

Advantages:

- Natural hierarchical organization.
- Supports grouping by project, user, type, etc.
- Easy path-based naming.
- Scales well.

Disadvantages:

- Path traversal takes time.
- Deleting directories must be handled carefully.
- Sharing may require links or permissions.

Most modern systems use a tree-structured directory model.

Acyclic Graph Directory

An acyclic graph directory allows sharing of files or directories using links, but cycles are not allowed.

Example:

```
/project/report.txt
/home/user/report-link -> /project/report.txt
```

Advantages:

- Supports sharing.
- Avoids duplication.
- Saves space.

Disadvantages:

- Deletion becomes more complex.
- Reference counting may be needed.
- Links can become dangling.

General Graph Directory

A general graph directory allows cycles.

Example problem:

```
A links to B
B links back to A
```

This can create infinite traversal loops.

Advantages:

- Maximum flexibility.

Disadvantages:

- Cycles complicate traversal.
- Garbage collection may be needed.
- Backup and search algorithms must detect visited directories.

Because of these complications, file systems usually restrict hard links to directories.

Hard Links and Soft Links

Hard Link

A hard link is another directory entry pointing to the same file metadata object.

In Unix-like systems:

```
name1 -> inode 100
name2 -> inode 100
```

Both names refer to the same file.

If one name is deleted, the file remains as long as another hard link exists.

Characteristics:

- Points directly to inode.
- Cannot usually cross file system boundaries.
- Usually not allowed for directories.
- File is removed only when link count becomes zero and no process has it open.

Soft Link or Symbolic Link

A symbolic link is a special file containing the path of another file.

```
shortcut -> /home/student/file.txt
```

Characteristics:

- Points to pathname, not directly to inode.
- Can cross file system boundaries.
- Can point to directories.
- Can become dangling if target is deleted.

Comparison:

Feature	Hard Link	Symbolic Link
Points to	Inode/file metadata	Pathname
Crosses file systems	Usually no	Yes
Can become dangling	No, while inode exists	Yes
Can link directories	Usually restricted	Yes
Has separate inode	Directory entry points to same inode	Symmlink has its own inode

File System Mounting

Mounting is the process of attaching a file system to the existing directory tree.

In Unix-like systems, there is one root directory `/`. Other file systems are mounted at directories called mount points.

Example:

```
mount /dev/sdb1 /mnt/usb
```

After mounting, files from `/dev/sdb1` become accessible under `/mnt/usb`.

Mount Point

A mount point is a directory where another file system is attached.

Example:

```
/
|-- home
|-- mnt
|   |-- usb
```

If a USB drive is mounted at `/mnt/usb`, accessing `/mnt/usb/photos` reads from the USB file system.

Windows Drive Letters

Windows often exposes mounted volumes using drive letters:

```
C:\  
D:\  
E:\
```

This is different from Unix's single-root tree model.

Why Mounting Matters

Mounting allows:

- Multiple storage devices to appear in one namespace.
- Removable drives to be attached and detached.
- Network file systems to be accessed like local directories.
- Different file systems to coexist.

Disk Blocks and File Allocation

Storage devices are divided into fixed-size units.

Important terms:

Term	Meaning
Sector	Smallest physical addressable unit on many disks
Block	File system allocation unit, often multiple sectors
Cluster	Allocation unit in some file systems, especially FAT/NTFS terminology
Page	Unit often used in memory or flash storage

File systems usually allocate disk space in blocks.

Example:

If block size is 4 KB and file size is 10 KB:

- Block 1 stores 4 KB.
- Block 2 stores 4 KB.
- Block 3 stores remaining 2 KB.
- 2 KB inside the third block is wasted as internal fragmentation.

Logical vs Physical View

User view:

```
file.txt = bytes 0 to 9999
```

File system view:

```
file.txt -> block 10, block 11, block 205
```

Disk view:

```
physical sectors/pages on storage device
```

Allocation Problem

The file system must decide:

- Which blocks should be assigned to a file?
- How should it remember those block locations?
- How can files grow?
- How can space be reused after deletion?
- How can fragmentation be reduced?

Main allocation methods:

- Contiguous allocation
- Linked allocation
- Indexed allocation

Contiguous Allocation

In contiguous allocation, each file occupies a set of consecutive disk blocks.

Example:

```
File A starts at block 10, length = 5  
File A uses blocks 10, 11, 12, 13, 14
```

Directory entry stores:

```
Starting block + length
```

Example

File	Start Block	Length
A	10	5
B	30	3
C	50	8

File A uses blocks:

```
10, 11, 12, 13, 14
```

Advantages

- Very simple.
- Excellent sequential access.
- Excellent random access.
- Minimal metadata.
- Few disk seeks if blocks are physically close.

Random access formula:

```
physical block = start block + logical block number
```

If a file starts at block 100 and the program wants logical block 7:

```
physical block = 100 + 7 = 107
```

Disadvantages

- External fragmentation.
- File growth is difficult.
- Need to know file size in advance.
- Compaction may be required.

External Fragmentation

External fragmentation happens when free space is split into small pieces scattered across disk.

Example:

```
Free blocks: 5 blocks here, 3 blocks there, 7 blocks elsewhere
```

Total free space may be enough for a 10-block file, but no single contiguous 10-block region may exist.

File Growth Problem

Suppose file A occupies blocks 10 to 14.

If block 15 is already used by another file, file A cannot grow directly.

Possible solutions:

- Move file A to a larger contiguous region.
- Reserve extra space during allocation.
- Use extents.

Extents

An extent is a contiguous group of blocks.

Modern file systems often use extent-based allocation instead of pure contiguous allocation.

Example:

```
File A -> extent 1: start 10, length 5  
        -> extent 2: start 80, length 4
```

This combines benefits:

- Mostly contiguous storage.
- Better support for file growth.
- Less metadata than one pointer per block.

File systems such as ext4 and NTFS use extent-like ideas.

Linked Allocation

In linked allocation, each file is a linked list of disk blocks.

Each block contains:

- File data.
- Pointer to the next block.

Directory entry stores:

```
First block pointer
```

Sometimes it also stores the last block pointer for efficient appending.

Example

```
File A: 9 -> 16 -> 1 -> 25 -> EOF
```

The file's blocks can be anywhere on disk.

Advantages

- No external fragmentation.
- File can grow easily.
- Directory entry needs only starting block.
- Any free block can be used.

Disadvantages

- Poor random access.
- Pointer overhead inside each block.
- Reliability problem if a pointer is corrupted.
- Blocks scattered across disk can cause many seeks.

Random Access Problem

To access logical block 100, the file system must follow pointers:

```
block 0 -> block 1 -> block 2 -> ... -> block 100
```

This is slow.

Pointer Overhead

If each block is 4096 bytes and pointer size is 4 bytes, then only 4092 bytes remain for data.

This wastes space and makes block size awkward.

Reliability Problem

If one block pointer is damaged, the rest of the file may become unreachable.

Example:

```
A -> B -> C -> D
```

If pointer in B is corrupted, C and D may be lost from the chain.

FAT: File Allocation Table

FAT stands for File Allocation Table.

It is a variation of linked allocation.

Instead of storing the next-block pointer inside each data block, FAT stores all next-block pointers in a separate table.

Basic Idea

There is one table entry for each disk block or cluster.

Example:

```
FAT[9]  = 16  
FAT[16] = 1  
FAT[1]  = 25  
FAT[25] = EOF
```

This means:

```
File A: 9 -> 16 -> 1 -> 25 -> EOF
```

Directory entry stores the first block number.

Advantages

- Data blocks contain only data, not pointers.
- Easier to traverse file chains if FAT is cached in memory.
- Simple to implement.
- Used historically in MS-DOS, FAT16, FAT32, and removable devices.

Disadvantages

- FAT can become very large.
- Random access still requires following the chain.
- If FAT is corrupted, many files can be affected.
- Not ideal for very large disks.

FAT Entry Values

A FAT entry may indicate:

- Next cluster number.
- End of file.
- Free cluster.
- Bad cluster.
- Reserved cluster.

FAT32

FAT32 is still common on USB drives and memory cards because:

- It is simple.
- It has wide OS compatibility.
- It is supported by many devices.

Limitations:

- Maximum single file size is usually 4 GB minus 1 byte.
 - No strong permissions model.
 - No journaling.
 - Less reliable than modern journaling file systems.
-

Indexed Allocation

In indexed allocation, each file has an index block containing pointers to its data blocks.

Directory entry points to the index block.

Example

```
Directory entry for File A -> Index block 20
```

```
Index block 20:
```

```
0 -> 9  
1 -> 16  
2 -> 1  
3 -> 25
```

This means:

```
Logical block 0 is physical block 9  
Logical block 1 is physical block 16  
Logical block 2 is physical block 1  
Logical block 3 is physical block 25
```

Advantages

- Supports direct/random access.
- No external fragmentation.
- File can grow by adding pointers.
- Data blocks do not need pointer overhead.

Disadvantages

- Index block overhead.
- Small files may waste space if a whole index block is allocated.
- Very large files may need multi-level indexing.

Random Access

To access logical block **i**, the file system checks index entry **i**.

Example:

```
logical block 2 -> index[2] -> physical block 1
```

This is much faster than linked allocation for random access.

Large File Problem

If one index block can hold only 1024 pointers, then it can address only 1024 data blocks.

For larger files, systems use:

- Linked index blocks.
- Multi-level indexing.
- Combined direct and indirect pointers.

Unix inodes use the combined direct/indirect approach.

Inodes

An inode is a data structure used by Unix-like file systems to store metadata about a file.

The word inode means index node.

Important:

An inode stores file metadata and block pointers, but usually does not store the filename.

The filename is stored in a directory entry.

What an Inode Contains

An inode commonly stores:

- File type.
- File permissions.
- Owner user ID.
- Group ID.
- File size.
- Timestamps.
- Link count.

- Pointers to data blocks.
- Pointers to indirect blocks.

An inode usually does not contain:

- The file name.
- The full path.

Directory Entry and Inode Relationship

In Unix-like systems:

```
Directory entry = filename + inode number
Inode = metadata + block pointers
```

Example:

```
/home/student/notes.txt

Directory "student" contains:
notes.txt -> inode 3057

Inode 3057 contains:
size, owner, permissions, timestamps, block pointers
```

Why Filename Is Not in the Inode

Because multiple filenames can point to the same inode using hard links.

Example:

```
a.txt -> inode 100
b.txt -> inode 100
```

If the filename were stored inside the inode, one inode could not naturally support multiple names.

Inode Number

Each inode has a unique inode number within a file system.

The pair:

```
file system + inode number
```

uniquely identifies a file.

Link Count

The link count stores the number of directory entries pointing to the inode.

Example:

```
ln a.txt b.txt
```

Now both `a.txt` and `b.txt` point to the same inode, and link count becomes 2.

When a filename is deleted:

- Link count decreases.
- If link count becomes 0 and no process has the file open, the inode and data blocks are freed.

Inode Block Pointers

Traditional Unix inodes contain:

- Direct block pointers.
- Single indirect pointer.
- Double indirect pointer.
- Triple indirect pointer.

This design supports both small and very large files efficiently.

Direct Pointers

Direct pointers point directly to data blocks.

Example:

```
inode direct[0] -> data block 100  
inode direct[1] -> data block 105
```

Direct pointers are fast because no extra lookup block is needed.

They are good for small files.

Single Indirect Pointer

A single indirect pointer points to a block that contains many data block addresses.

```
inode -> indirect block -> data blocks
```

This supports larger files.

Double Indirect Pointer

A double indirect pointer points to a block containing pointers to indirect blocks.

```
inode -> double indirect block -> indirect blocks -> data blocks
```

This supports much larger files.

Triple Indirect Pointer

A triple indirect pointer adds one more level.

```
inode -> triple indirect block -> double indirect blocks -> indirect blocks -> data blocks
```

This supports extremely large files.

Example Calculation

Suppose:

- Block size = 4 KB.
- Block pointer size = 4 bytes.
- Each block can store $4096 / 4 = 1024$ pointers.
- Inode has 12 direct pointers, 1 single indirect, 1 double indirect, 1 triple indirect.

Maximum data blocks:

```
Direct = 12
Single indirect = 1024
Double indirect = 1024 * 1024
Triple indirect = 1024 * 1024 * 1024
```

Total addressable data is enormous.

This design is efficient because small files use direct pointers, while large files use indirect blocks only when needed.

Inode Exhaustion

A file system can run out of inodes even if disk space is still available.

This happens when there are too many small files.

Example:

```
Disk space free: 20 GB
Free inodes: 0
```

In that case, new files cannot be created because every file needs an inode.

Inode vs File Descriptor

Concept	Meaning
Inode	Persistent file metadata on disk
File descriptor	Per-process handle for an open file

An inode exists even when no process has the file open.

A file descriptor exists only inside a running process.

Inode vs Filename

Concept	Meaning
Filename	Human-readable name in a directory
Inode	Internal metadata object

Multiple filenames can refer to the same inode.

Free-Space Management

Free-space management is how the file system tracks which disk blocks are free and which are allocated.

The file system must quickly answer:

- Which blocks are available for a new file?
- Which blocks can be reused after deletion?
- Is there enough free space?
- Where should new blocks be allocated to reduce fragmentation?

Common methods:

- Bit vector or bitmap.
- Linked list.
- Grouping.
- Counting.
- Space maps or extent trees.

Bitmap or Bit Vector

A bitmap uses one bit per block.

Example:

```
1 = free  
0 = allocated
```

or the opposite convention, depending on the file system.

Example:

```
Block:  0 1 2 3 4 5 6 7  
Bitmap: 1 0 0 1 1 0 1 0
```

If **1** = **free**, then blocks 0, 3, 4, and 6 are free.

Advantages of Bitmap

- Simple.
- Compact.
- Easy to find contiguous free blocks using bit scanning.
- Good for modern file systems.

Disadvantages of Bitmap

- Bitmap itself can be large for very large disks.
- Must keep bitmap consistent with actual allocation.
- Searching can be slow if disk is nearly full, unless optimized.

Bitmap Size Calculation

Suppose:

- Disk size = 1 GB.
- Block size = 4 KB.

Number of blocks:

```
1 GB / 4 KB = 2^30 / 2^12 = 2^18 = 262144 blocks
```

Bitmap size:

```
262144 bits = 32768 bytes = 32 KB
```

So only 32 KB is needed to track 1 GB of disk with 4 KB blocks.

Linked List of Free Blocks

In this method, all free blocks are linked together.

Example:

```
free list head -> block 5 -> block 19 -> block 44 -> block 8 -> null
```

Each free block stores a pointer to the next free block.

Advantages

- Simple.
- No separate large bitmap needed.
- Easy to allocate one block from the head.

Disadvantages

- Hard to find contiguous free blocks.
- Traversing the list can be slow.
- Extra disk reads may be required.
- Pointer corruption can damage free-space tracking.

Grouping

Grouping improves the linked-list approach.

Instead of each free block pointing to only one next block, the first free block stores addresses of many other free blocks.

Example:

```
Free block A stores:  
20, 21, 22, 35, 40, next_group_block
```

This allows the system to learn about many free blocks with one disk read.

Counting

Counting stores a starting block and a count of consecutive free blocks.

Example:

```
Start = 100, Count = 20
```

This means blocks 100 to 119 are free.

Advantages:

- Efficient when free space occurs in runs.
- Good for extent-based allocation.
- Less metadata than listing every block.

Disadvantages:

- Less useful if free space is highly fragmented.

Extent-Based Free-Space Management

Modern file systems often track free space as extents.

An extent is:

```
start block + length
```

Example:

```
Free extents:  
start 100, length 50  
start 500, length 120  
start 900, length 10
```

This is efficient for large disks because contiguous ranges are common.

Free-Space Management Issues

Key challenges:

- Avoiding fragmentation.
- Allocating nearby blocks for the same file.
- Updating free-space metadata safely after crashes.
- Scaling to large disks.
- Handling concurrent allocations.

Journaling

Journaling is a technique used by file systems to improve crash recovery.

The basic idea:

```
Before making important changes to the file system, record the intended changes in a special area called a journal or log.
```

If the system crashes, the file system can inspect the journal and complete or undo operations to restore consistency.

Why Journaling Is Needed

File system operations often involve multiple updates.

Example: creating a file may require:

1. Allocate an inode.
2. Allocate data blocks.
3. Update free-space bitmap.
4. Add directory entry.
5. Update directory metadata.
6. Update inode metadata.

If a crash happens in the middle, the file system can become inconsistent.

Example inconsistency:

- Directory entry points to an inode that was not properly initialized.
- A block is marked allocated but no file points to it.
- A file points to a block marked free.
- File size says 8 KB but only one 4 KB block was allocated.

Journaling reduces these problems.

Journal

A journal is a reserved area on disk that stores transaction records.

A transaction is a group of updates that should happen atomically.

Atomic means:

Either all updates happen, or none of them appear to happen.

Basic Journaling Flow

For a metadata update:

1. Write a description of the planned changes to the journal.
2. Mark the journal transaction as committed.
3. Apply the actual changes to the file system.
4. Remove or mark the journal entry as completed.

If a crash occurs:

- If transaction was not committed, ignore it.
- If transaction was committed but not fully applied, replay it.

Write-Ahead Logging

Journaling usually follows write-ahead logging:

The journal record must reach stable storage before the actual file system metadata is modified.

This ensures recovery has enough information after a crash.

Metadata Journaling

Only metadata changes are journaled.

Metadata includes:

- Inodes.
- Directory entries.
- Free-space bitmap.
- Allocation structures.

User file data may not be journaled.

Advantages:

- Less overhead than full data journaling.
- Protects file system structure.
- Common default in many systems.

Disadvantages:

- File system structure remains consistent, but recently written file data may be lost or contain old data after crash.

Data Journaling

Both metadata and file data are written to the journal.

Advantages:

- Stronger consistency.
- Better protection for file contents.

Disadvantages:

- More disk writes.
- Slower performance.
- Journal requires more space.

Ordered Journaling

Ordered journaling is a common compromise.

Idea:

- File data is written to disk before metadata that points to it is committed.
- Metadata is journaled.

- File data itself is not necessarily journaled.

This prevents metadata from pointing to uninitialized garbage data.

Journaling Modes

Mode	What is journaled?	Pros	Cons
Writeback	Metadata only, weak ordering	Fast	Data may be stale after crash
Ordered	Metadata journaled, data written before metadata commit	Good balance	Some data loss still possible
Data journaling	Metadata and data	Strong consistency	Slowest

Journaling vs Backup

Journaling is not backup.

Journaling helps recover consistency after crashes.

It does not protect against:

- Accidental deletion.
- Disk failure.
- Malware.
- User overwriting a file.
- Long-term corruption.

For that, backups are needed.

Journaling vs fsck

fsck means file system check.

Without journaling, after a crash the OS may need to scan the whole file system to find inconsistencies.

With journaling, recovery is faster because the OS usually only needs to inspect the journal.

Comparison:

Feature	fsck-only recovery	Journaling recovery
Recovery speed	Slow for large disks	Usually fast
Scope	Scans many structures	Replays journal
Complexity	Simpler FS design	More complex FS
Consistency	Can repair	Can prevent many inconsistencies

Examples of Journaling File Systems

Examples:

- ext3
- ext4
- NTFS
- XFS
- APFS
- HFS+

FAT32 does not support journaling.

Permissions and Protection

Permissions control who can access files and what operations they can perform.

Protection is necessary because:

- Multiple users may share a system.
- Processes should not access unauthorized data.
- Programs should not accidentally damage system files.
- Malware impact should be limited.

Access Rights

Common access rights:

Permission	Meaning for File	Meaning for Directory
Read	Read file contents	List directory entries
Write	Modify file contents	Create, delete, or rename entries inside directory
Execute	Run file as program	Traverse/search through directory

The directory meaning is very important for interviews.

Unix Permission Model

Unix-like systems use three permission classes:

- User/owner
- Group
- Others

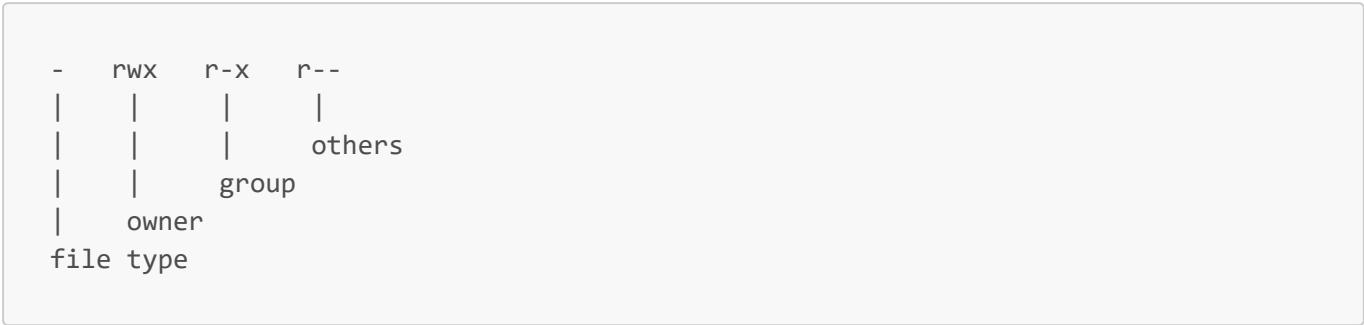
Each class can have:

- Read **r**
- Write **w**
- Execute **x**

Example:

```
-rwxr-xr--
```

Breakdown:



Meaning:

- Owner can read, write, execute.
- Group can read and execute.
- Others can only read.

Numeric Permissions

Permissions are often represented using octal numbers.

Permission	Value
Read	4
Write	2
Execute	1

Examples:

Octal	Symbolic	Meaning
7	rwx	read + write + execute
6	rw-	read + write
5	r-x	read + execute
4	r--	read only
0	---	no permission

Full examples:

Mode	Symbolic	Meaning
755	rwxr-xr-x	Owner full, group/others read+execute
644	rw-r--r--	Owner read/write, group/others read

Mode	Symbolic	Meaning
700	rwX-----	Only owner has full access
600	rw-----	Only owner can read/write

chmod

`chmod` changes permissions.

Examples:

```
chmod 755 script.sh
chmod u+x script.sh
chmod go-r file.txt
```

chown and chgrp

`chown` changes owner.

```
chown alice file.txt
```

`chgrp` changes group.

```
chgrp developers file.txt
```

Directory Permission Examples

For directories:

- Read permission allows listing filenames.
- Execute permission allows entering/traversing the directory.
- Write permission allows modifying entries, usually only useful with execute.

Important cases:

Directory Permission	Effect
<code>r--</code>	Can list names only if traversal is already possible; often not useful alone
<code>--x</code>	Can access known filenames but cannot list directory
<code>r-x</code>	Can list and enter directory
<code>rwX</code>	Can list, enter, create, delete, rename

Example:

If you have execute but not read on a directory, you cannot list its contents, but you can access a file if you already know its name.

Access Control Lists

Traditional Unix permissions are limited to owner, group, and others.

Access Control Lists, or ACLs, allow more fine-grained permissions.

Example:

```
User Alice: read/write
User Bob: read only
Group developers: read/write
Others: no access
```

Advantages:

- More flexible.
- Supports multiple users and groups with different permissions.

Disadvantages:

- More complex to manage.
- Harder to audit manually.

Windows Permissions

Windows NTFS uses ACL-based permissions.

Common permissions:

- Full control.
- Modify.
- Read and execute.
- List folder contents.
- Read.
- Write.

NTFS permissions can be inherited from parent directories.

Special Unix Permission Bits

Unix-like systems have special permission bits:

- setuid
- setgid
- sticky bit

setuid

If `setuid` is set on an executable, the program runs with the file owner's privileges.

Example:

The `passwd` command needs to update password-related system files, so it may run with elevated privileges.

Risk:

If a `setuid` program has a vulnerability, attackers may gain extra privileges.

setgid

If `setgid` is set on an executable, the program runs with the file group's privileges.

On a directory, `setgid` causes newly created files to inherit the directory's group.

Useful for shared project directories.

Sticky Bit

The sticky bit is often used on shared directories such as `/tmp`.

It means:

Users can create files in the directory, but they can delete only their own files, unless they are root or own the directory.

Example:

```
ls -ld /tmp
```

You may see:

```
drwxrwxrwt
```

The `t` indicates sticky bit.

umask

`umask` defines default permission bits to remove when new files or directories are created.

Common default:

```
umask 022
```

If default file mode is 666:

```
666 - 022 = 644
```

If default directory mode is 777:

```
777 - 022 = 755
```

Files usually do not get execute permission by default.

Permission Checking

When a process tries to access a file, the OS checks:

1. User identity of the process.
2. Group memberships.
3. File ownership.
4. Permission bits or ACLs.
5. Requested operation.

Example:

If a process wants to write to `report.txt`, the OS checks whether the process's user has write permission through owner, group, others, or ACL.

File System Implementation Layers

File systems are usually implemented in layers.

Application Layer

User programs call library functions.

Example:

```
fopen("data.txt", "r");
```

System Call Interface

Library functions call OS system calls.

Examples:

```
open()  
read()
```

```
write()  
close()
```

Virtual File System

A Virtual File System, or VFS, provides a common interface for many file systems.

For example, Linux can support ext4, FAT32, NTFS, tmpfs, procfs, and NFS through a VFS layer.

VFS allows the OS to treat different file systems uniformly.

File System-Specific Layer

This layer knows the details of a specific file system.

Examples:

- ext4 allocation rules.
- NTFS Master File Table.
- FAT table traversal.
- XFS extent management.

Buffer Cache / Page Cache

Caches disk blocks in memory to reduce disk I/O.

Benefits:

- Faster repeated reads.
- Write buffering.
- Read-ahead.
- Fewer physical disk operations.

Block Device Driver

The device driver communicates with the storage hardware.

It handles:

- Submitting read/write requests.
- Device queues.
- Hardware-specific commands.

Storage Hardware

The actual device:

- HDD
- SSD
- USB drive
- NVMe drive

- Network storage

Layered View

```
Application
|
C library
|
System calls
|
Virtual File System
|
Specific file system implementation
|
Buffer/page cache
|
Block device driver
|
Storage device
```

Reliability and Recovery

File systems must protect data against:

- Sudden power loss.
- Kernel crash.
- Hardware failure.
- Bad sectors.
- Software bugs.
- Interrupted writes.

Consistency

A file system is consistent when its metadata structures agree with each other.

Examples of consistency:

- Every allocated block belongs to a file or metadata structure.
- A free block is not referenced by a file.
- Directory entries point to valid inodes.
- Inode link counts match actual directory references.

Common Inconsistencies

After a crash, possible inconsistencies include:

- Lost blocks.
- Duplicate block allocation.
- Incorrect file size.

- Incorrect link count.
- Directory entry pointing to invalid inode.
- Free-space bitmap mismatch.

fsck

fsck checks and repairs file system inconsistencies.

Typical checks:

- Validate superblock.
- Check free block lists or bitmaps.
- Check inode structure.
- Verify directory entries.
- Recalculate link counts.
- Detect duplicate block references.

Disadvantage:

On large disks, full scanning can take a long time.

Superblock

The superblock stores important metadata about the whole file system.

It may include:

- File system type.
- Size of file system.
- Block size.
- Number of blocks.
- Number of free blocks.
- Number of inodes.
- Number of free inodes.
- Location of important metadata structures.
- Mount state.

Because it is critical, file systems often keep backup copies of the superblock.

Bad Block Management

Bad blocks are storage blocks that cannot reliably store data.

Handling methods:

- Mark bad blocks so they are not allocated.
- Use spare blocks to replace bad ones.
- Let disk firmware remap bad sectors internally.

Modern drives usually handle many bad sectors inside firmware, but file systems and OS tools may still detect and respond to failures.

Performance Considerations

File system performance depends on many factors:

- Disk type: HDD or SSD.
- Block size.
- Allocation method.
- Fragmentation.
- Caching.
- Journaling mode.
- Directory structure.
- Workload type.

Caching

The OS uses memory to cache recently used file data and metadata.

If data is found in cache, disk access is avoided.

This greatly improves performance because memory is much faster than disk.

Buffering

Writes are often buffered.

Instead of immediately writing every small update to disk, the OS stores changes in memory and writes them later.

Advantages:

- Combines small writes.
- Reduces disk I/O.
- Improves throughput.

Risk:

- Data may be lost if power fails before buffers are flushed.

Commands such as `sync` or system calls such as `fsync()` force data to stable storage.

Read-Ahead

If the OS detects sequential reading, it may prefetch upcoming blocks before the process asks for them.

Useful for:

- Video playback.
- Reading large files.
- Copying files.

Write-Behind

The OS may delay writes and perform them in the background.

This improves performance but can increase data-loss risk if not carefully managed.

Fragmentation

Fragmentation occurs when a file's blocks are scattered across disk.

For HDDs:

- Fragmentation causes more seek time.
- Performance can degrade significantly.

For SSDs:

- Fragmentation is less harmful because there is no mechanical seek.
- But it can still increase metadata overhead and reduce sequential I/O efficiency.

Block Size Trade-Off

Large block size:

- Better sequential performance.
- Smaller metadata overhead.
- More internal fragmentation for small files.

Small block size:

- Less internal fragmentation.
- More metadata overhead.
- Potentially more I/O operations for large files.

Locality

Good file systems try to place related data close together.

Examples:

- Store a file's blocks near its inode.
- Store files in the same directory near each other.
- Allocate large extents for large files.

This improves performance, especially on HDDs.

Common File Systems

FAT32

Characteristics:

- Simple.
- Widely supported.
- Common on USB drives.
- Uses File Allocation Table.

- No journaling.
- Limited permissions.
- Maximum file size around 4 GB.

Good for:

- Compatibility.
- Small removable drives.

Not ideal for:

- Large files.
- Secure multi-user systems.
- Crash consistency.

exFAT

exFAT was designed for flash drives and removable storage.

Characteristics:

- Supports larger files than FAT32.
- Better for large removable drives.
- Widely supported by modern systems.
- Simpler than NTFS.

Good for:

- SD cards.
- USB drives.
- Cross-platform removable storage.

NTFS

NTFS is the main Windows file system.

Characteristics:

- Journaling.
- Access control lists.
- Compression support.
- Encryption support.
- Large file support.
- Uses Master File Table, or MFT.

Good for:

- Windows system drives.
- Secure multi-user environments.
- Large disks and files.

ext4

ext4 is a widely used Linux file system.

Characteristics:

- Journaling.
- Extent-based allocation.
- Backward evolution from ext2/ext3.
- Good performance and reliability.
- Supports large files and volumes.

Good for:

- Linux desktops.
- Servers.
- General-purpose workloads.

XFS

XFS is a high-performance journaling file system.

Characteristics:

- Excellent for large files.
- Extent-based.
- Scales well.
- Often used on servers.

Good for:

- Large storage systems.
- High-throughput workloads.

APFS

APFS is Apple's modern file system.

Characteristics:

- Designed for SSDs.
- Copy-on-write metadata.
- Snapshots.
- Clones.
- Encryption support.

Good for:

- macOS and iOS devices.
- SSD-based Apple systems.

Btrfs

Btrfs is a Linux copy-on-write file system.

Characteristics:

- Snapshots.
- Checksums.
- Subvolumes.
- Copy-on-write.
- Built-in volume management features.

Good for:

- Snapshot-heavy systems.
- Advanced Linux storage use cases.

Important Interview Comparisons

Contiguous vs Linked vs Indexed Allocation

Feature	Contiguous	Linked	Indexed
Allocation style	Consecutive blocks	Blocks linked by pointers	Index block stores pointers
Sequential access	Excellent	Good if chain traversal is okay	Good
Random access	Excellent	Poor	Good
External fragmentation	Yes	No	No
File growth	Difficult	Easy	Easier
Metadata overhead	Low	Pointer per block	Index block
Reliability	Good if metadata intact	Pointer corruption can break chain	Index corruption affects file
Best for	Static files, sequential data	Simple dynamic files	General-purpose file systems

Bitmap vs Linked Free List

Feature	Bitmap	Linked Free List
Representation	One bit per block	Free blocks linked together
Space efficiency	Very compact	Uses pointers in free blocks
Finding contiguous blocks	Good	Poor
Allocation of one block	Good	Very easy
Reliability	Bitmap corruption dangerous	Pointer corruption dangerous

Feature	Bitmap	Linked Free List
Modern use	Common	Less common alone

Hard Link vs Symbolic Link

Feature	Hard Link	Symbolic Link
Points to	Same inode	Pathname
Separate inode	No	Yes
Works across file systems	Usually no	Yes
Can point to deleted target	Not in normal sense	Yes, dangling symlink
Directory linking	Usually restricted	Allowed
Link count changes	Yes	Target link count usually unchanged

Inode vs Directory Entry

Feature	Inode	Directory Entry
Stores filename	Usually no	Yes
Stores metadata	Yes	Minimal
Stores block pointers	Yes	No
Used for hard links	Multiple entries can point to same inode	Yes
Persistent	Yes	Yes

Journaling vs Copy-on-Write

Feature	Journaling	Copy-on-Write
Basic idea	Log changes before applying them	Write new version elsewhere, then switch pointer
Recovery	Replay or discard journal transactions	Use consistent old or new version
Write overhead	Journal writes plus actual writes	New block writes and metadata updates
Examples	ext3, ext4, NTFS	Btrfs, ZFS, APFS metadata

Frequently Asked Placement Questions

1. What is a file system?

A file system is an OS component that organizes data on storage devices into files and directories. It manages naming, storage allocation, metadata, permissions, free space, and crash recovery.

2. What is the difference between a file and a directory?

A file stores data. A directory is a special file that stores mappings from names to file metadata references, such as inode numbers.

3. What happens when a file is opened?

The OS checks permissions, locates the file metadata, creates entries in open-file tables, initializes the current file offset, records the access mode, and returns a file descriptor or handle to the process.

4. Why is open needed before read or write?

Opening avoids repeated path lookup, lets the OS verify permissions once, creates kernel data structures for the file, and gives the process a handle for future operations.

5. What is an inode?

An inode is a Unix-like file system data structure that stores file metadata and block pointers. It usually does not store the filename.

6. Why does an inode not contain the filename?

Because multiple directory entries can point to the same inode through hard links. If the name were stored in the inode, one inode could not naturally have multiple names.

7. What is the difference between hard link and soft link?

A hard link points directly to the same inode. A symbolic link is a separate file containing the target path. Hard links usually cannot cross file systems, while symbolic links can.

8. What is contiguous allocation?

Contiguous allocation stores each file in consecutive disk blocks. It gives excellent sequential and random access but suffers from external fragmentation and difficult file growth.

9. What is linked allocation?

Linked allocation stores a file as a linked list of disk blocks. It avoids external fragmentation and supports easy growth, but random access is slow and pointer corruption can be dangerous.

10. What is indexed allocation?

Indexed allocation uses an index block containing pointers to a file's data blocks. It supports random access and avoids external fragmentation, but needs extra index storage.

11. What is FAT?

FAT, or File Allocation Table, is a linked-allocation-based file system structure where block chain pointers are stored in a central table instead of inside data blocks.

12. What is free-space management?

Free-space management is the file system's method for tracking which disk blocks are free and which are allocated. Common methods include bitmaps, linked lists, grouping, counting, and extents.

13. What is a bitmap in file systems?

A bitmap uses one bit per disk block to indicate whether that block is free or allocated. It is compact and good for finding contiguous free blocks.

14. What is journaling?

Journaling records planned file system changes in a log before applying them. After a crash, the file system uses the journal to restore consistency quickly.

15. Does journaling prevent data loss?

Not completely. Journaling mainly protects file system consistency. Depending on the journaling mode, recently written file data may still be lost after a crash.

16. What is the difference between metadata journaling and data journaling?

Metadata journaling logs only file system metadata. Data journaling logs both metadata and file contents. Data journaling gives stronger consistency but is slower.

17. What does execute permission mean for a directory?

Execute permission on a directory means the user can traverse or search through the directory. Without execute permission, the user cannot access files inside even if they know the names.

18. What is the sticky bit?

The sticky bit on a directory allows users to create files there but prevents them from deleting files owned by other users. It is commonly used on `/tmp`.

19. What is external fragmentation?

External fragmentation occurs when free disk space is split into small non-contiguous pieces, making it hard to allocate a large contiguous file even if total free space is enough.

20. What is internal fragmentation?

Internal fragmentation occurs when allocated space inside a block is unused. For example, a 1 KB file stored in a 4 KB block wastes 3 KB.

21. Why are extents used?

Extents represent a range of contiguous blocks using start block and length. They reduce metadata overhead and improve performance for large files.

22. What is a superblock?

A superblock stores metadata about the entire file system, such as block size, file system size, inode count, free block count, and locations of important structures.

23. What happens when a file is deleted in Unix?

The directory entry is removed and the inode link count is decremented. If the link count becomes zero and no process has the file open, the inode and data blocks are freed.

24. Can a file be deleted while a process is using it?

In Unix-like systems, yes. The filename can be removed, but the file data remains accessible to processes that already have it open until the last file descriptor is closed.

25. Why can a disk have free space but still fail to create files?

Because the file system may have run out of inodes. Each new file needs an inode, so inode exhaustion prevents file creation even if data blocks are available.

Quick Revision Sheet

Core Definitions

Term	Meaning
File system	OS structure for organizing files and directories on storage
File	Named collection of data
Directory	Mapping from names to file references
Metadata	Data about a file
Inode	Unix metadata structure for a file
File descriptor	Process-level integer handle for an open file
Superblock	Metadata about the whole file system
Block	File system allocation unit
Extent	Contiguous range of blocks
Journal	Log used for crash recovery

File Operations

Operation	Purpose
Create	Make new file
Open	Prepare file for access
Read	Copy data from file to memory
Write	Copy data from memory to file
Seek	Change current file offset
Close	Release open file handle
Delete/unlink	Remove directory entry
Truncate	Change file size
Rename	Change file name/path

Allocation Methods

Method	Main Strength	Main Weakness
Contiguous	Fast access	External fragmentation
Linked	Easy growth	Slow random access
Indexed	Good random access	Index overhead

Free-Space Methods

Method	Key Idea
Bitmap	One bit per block
Linked list	Free blocks linked together
Grouping	Store many free block addresses in one block
Counting	Store start block and count
Extent map	Track free ranges

Unix Permission Values

Permission	Value
Read	4
Write	2
Execute	1

Common modes:

```
755 = rwxr-xr-x
644 = rw-r--r--
700 = rwx-----
600 = rw-----
```

Must-Remember Interview Lines

- A file system maps human-friendly filenames to physical storage blocks.
- A directory maps names to file metadata references.
- An inode stores metadata and block pointers, not usually the filename.
- Contiguous allocation is fast but suffers from external fragmentation.
- Linked allocation avoids external fragmentation but has poor random access.
- Indexed allocation supports random access using an index block.
- Bitmap free-space management is compact and good for finding contiguous blocks.
- Journaling improves crash recovery by logging changes before applying them.
- Journaling is not the same as backup.
- Execute permission on a directory means traversal permission.
- A hard link points to the same inode; a symbolic link points to a path.